



Universidad Autónoma de Baja California
Facultad de Ciencias Químicas e Ingeniería
ISTE

LPOO

Unidad 2
Clases

Una clase la podemos definir como el plano de construcción que va a utilizar en este caso para construir un objeto, **es por eso que se dice que un objeto es la instancia de una clase y que un objeto no puede existir sin una clase.**

Figure 6-1 A blueprint and houses built from the blueprint

Blueprint that describes a house



Instances of the house described by the blueprint



En otras palabras la clase describe el tipo de objeto en particular. Especifica la información que el objeto puede contener (**atributos o campos del objeto**) y las acciones que dicho objeto puede hacer (**métodos o comportamientos**), podemos ver que la clase es el plano de construcción para crear un objeto en particular.

Antes de programar un sistema/aplicación debemos realizar el diseño, uno de los diagramas que nos ayuda a visualizar la estructura de nuestra aplicación ya que **describe la clase, sus atributos, comportamientos y la relación entre objetos.**

Diagrama de clases (UML)

Nombre de la clase

Caja

Atributos

variable

-ancho : double

-largo : double

-depth : double ← tipo dato

acceso

Métodos

función

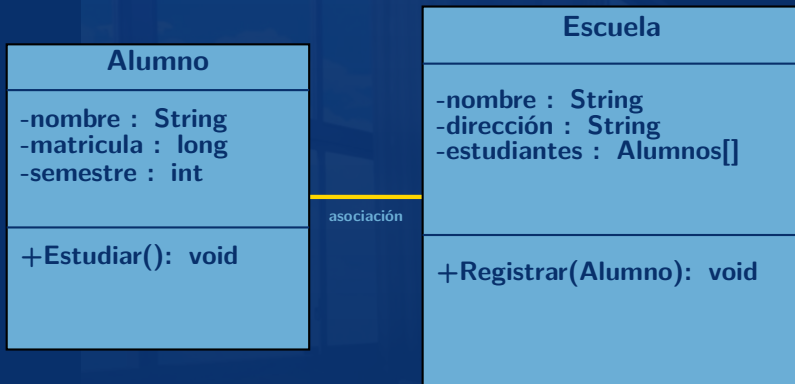
parámetros

+SetDimensiones(double,double,double)

+CalcularArea() : double

retorno

Diagrama de clases (UML)



El nombre de la clase es muy importante la razón principal es porque necesita que haya alta cohesión, El nombre debe describir lo que representa la clase.

Los atributos (variables de la clase) representan el estado, porque almacenan la información que el objeto guarda. (por ejemplo el nombre de la empresa, o la matricula del estudiante).

Estos pueden ser de cualquier tipo de dato primitivo así como de datos no primitivos (arreglos, **clases**)

Es importante que los atributos **se definan en privados** para conservar el principio de esconder lo más que se pueda la información.

Los métodos son las interfaces o protocolos para comunicar con otros objetos ó el comportamiento de la clase (lo que puede realizar).

Sintaxis una clase

```
1 [acceso] class [nombre] [extensiones]
2 {
3     // por defecto los atributos y métodos son públicos
4     // pero se recomienda que se use public o private o protected
5     [acceso] [tipo dato] atributo1
6     [acceso] [tipo dato] atributo2
7     [acceso] [tipo dato] atributoN
8     [acceso] [tipo dato] nombre_método1([parámetros])
9     {
10         //bloque de código del metodo1
11     }
12     [acceso] [tipo dato] nombre_método2([parámetros])
13     {
14         //bloque de código del metodo2
15     }
16     [acceso] [tipo dato] nombre_métodoN([parámetros])
17     {
18         //bloque de código de métodoN
19     }
20 }
```

Usando el diseño UML de la clase caja

Clase Caja

```
1 public class Caja
2 {
3     //Atributos
4     private double ancho;
5     private double largo;
6     private double depth;
7
8     // Métodos o comportamientos
9     public void SetDimensiones(double a, double l, double p)
10    {
11        ancho = a;
12        largo = l;
13        depth = p;
14    }
15    public double CalcularArea()
16    {
17        return ancho*largo*depth;
18    }
19 }
```

Se dice que un objeto es la instancia de una clase, esto es porque un objeto es una variable de **tipo de la clase que esta instanciando**

Para poder crear (instanciar) nuestro objeto es necesario usar la palabra reservada **new** esto hace que se reserve y se inicialice un espacio de memoria de tipo la clase que esta instanciando

hay que tener muy presente que para crear variables de cualquier tipo primitivo(int, char, float etc) **no es necesario utilizar la palabra new, sin embargo para instanciar una clase es necesario**

Usando la clase caja anterior, para poder crear un objeto se utiliza la palabra reservada **new**

```
1 class Demo
2 {
3     public static void main(String[] args)
4     {
5         Caja a = new Caja();
6         a.SetDimensiones(10,20,30);
7         System.out.println("El área de la caja es: "+a.CalcularArea());
8     }
9 }
10 }
```


Por defecto si no le ponemos un nivel de acceso a nuestras propiedades (atributos y métodos) se convierten en publicas dentro del paquete que se esta implementando o entorno que se encuentran.

El nivel de acceso significa que vamos a decidir quien tiene visibilidad a los atributos y métodos de una clase.

De manera estandarizada solamente **hay 3 niveles de acceso** pero pueden haber mas **dependiendo del lenguaje** que se este utilizando.

Este nivel de acceso se utiliza para que **cualquier otro objeto de otra clase pueda leer y modificar un atributo**, o para que **cualquier otro objeto de otra clase pueda invocar un método**.

Se representa con un **+** en UML a lado del atributo o del método.

usar public

```
1 class A
2 {
3     public int contador=0;
4 }
5 class B
6 {
7     //atributo de tipo clase A
8     A a;
9     public void UnMetodo()
10    {
11        a.contador++;
12        System.out.println(a.contador);
13    }
14 }
```

Este nivel de acceso se utiliza para **esconder la información que otros objetos de otras clases pueden ver.**

Solamente la clase que lo implementa tiene acceso y puede hacer modificaciones a los atributos y métodos.

Se representa con un - en UML a lado del atributo o del método.

private

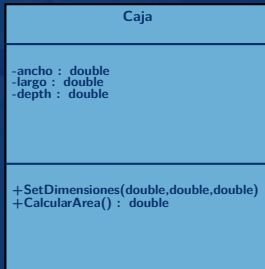
```
1 | class A
2 | {
3 |     private int contador=0;
4 |     public incrementarContador()
5 |     {
6 |         contador++;
7 |     }
8 |     final public int getContador()
9 |     {
10 |         return contador;
11 |     }
12 | }
13 | class B
14 | {
15 |     A a;
16 |     public void UnMetodo()
17 |     {
18 |         // no podemos acceder a la variable
19 |         // contador ya que es privada
20 |         // a.contador ++; //error
21 |         // se tiene que usar un método publico
22 |         a.incrementarContador();
23 |         System.out.println(a.getContador());
24 |     }
25 | }
```

Se comporta muy parecido al privado, con la única diferencia que **solamente objetos de clases que tengan una relación de padre-hijo pueden acceder y modificar a dichos atributos y métodos.**

Se representa con un **#** en UML a lado de los atributos o de los métodos .

Lo vamos a ver cuando lleguemos a herencia

Retomando el ejemplo de la clase Caja



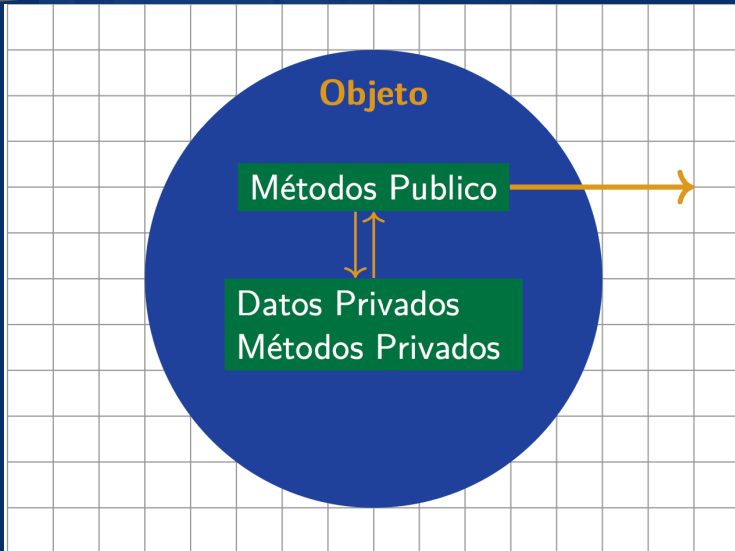
¿Porque el área no lo guardamos en un atributo?

Se dice que un atributo esta **"viejo" (stale)** cuando **depende de otros atributos**, como en este caso el área, depende de los otros atributos.

Si uno de los otros atributos cambia, el área **no cambiaría y se volvería un valor desactualizado**.

La encapsulación es como diseñamos los accesos a los atributos y métodos de nuestra clase para que los objetos de otras clases puedan utilizarlas y/o modificarlas.

En otras palabras La encapsulación se utiliza para **ocultar los valores o el estado de un objeto evitando el acceso directo a ellos** para evitar exponer detalles de Implementación..



Ejemplo Encapsulamiento

Caja.java

```
1 class Caja
2 {
3     //privados ya que vamos utilizar
4     //métodos publicos para acceder y modificar
5     private double ancho;
6     private double largo;
7     private double depth;
8
9     //los famosos getters de los atributos
10    final public double getAncho()
11    {
12        return ancho;
13    }
14    final public double getLargo()
15    {
16        return largo;
17    }
18    final public double getDepth()
19    {
20        return depth;
21    }
22    // el famoso setter aqui en este caso
23    // uno por los tres atributos
24    public void setDimensiones(double a, double l, double d)
25    {
26        ancho =a;
27        largo = l;
28        depth = d;
29    }
30    public double Area()
31    {
32        return ancho*largo*depth;
33    }
34    public String Info()
35    {
36        return "ancho: " +ancho+
37               " largo: "+largo+
38               " depth: "+depth+
39               "\narea: "+Area();
40    }
41 }
```

Demo.java

```
1 class Demo
2 {
3     public static void main(String[] args)
4     {
5         //creamos el objeto
6         Caja a = new Caja();
7         //asignamos valores a los atributos
8         a.setDimensiones(10,20,30);
9         //imprimimos la info de nuestra caja
10        System.out.println(a.Info());
11    }
12 }
```